SSL & TLS

Architecture

A technical blueprint of secure web communications, cryptographic handshakes, and the shift to Elliptic Curve efficiency.

The Foundation of Trust

What is SSL?

Secure Sockets Layer (SSL) is the legacy standard for establishing encrypted links between a server and client. Introduced in 1995, it relies heavily on RSA for key exchanges and is now considered deprecated due to security vulnerabilities.

What is TLS?

Transport Layer Security (TLS) is the modern, more secure successor to SSL. It fixes many SSL flaws, offers faster performance, and uses stronger encryption suites like AES-GCM and ChaCha20, keeping your data safe today.

The Safe & The Key

The Secret Handshake

Imagine sending a **Transparent Box** with a message inside. Without a lock, anyone can read it. SSL/TLS is the process of agreeing on a lock and key.

Step A: You send the server a padlock (Certificate).

Step B: The server shows you the key (Public Key).

Step C: You create a master key, hide it in a smaller box, and lock it with the server's padlock.

The CIA Triad Principles

Confidentiality

Ensures that sensitive information is accessible only to authorized parties. Achieved through robust encryption (AES/ChaCha20).

Integrity

Ensures that data is not altered in transit. Hashing (SHA-256/384) verifies that what was sent is exactly what was received.

Authentication

Verifies the identity of the communicating parties. Certificates prove that a website is who it claims to be.

Transport Layer & Ports

443

TCP Port

The Plumbing of Security

SSL/TLS operates on the **Transport Layer** of the OSI model. It primarily uses **TCP** to ensure reliable, ordered delivery of encrypted packets.

Port 443: The default standard for HTTPS (secure web traffic).

Port 465/587: Secure email (SMTPS).

Port 993/995: Secure mail retrieval (IMAPS/POP3S).

SSL Protocol Details

Symmetrical & Secret

- **Secret Key:** SSL creates a unique session key for every connection.
- **Cipher Selection:** The client and server "negotiate" an encryption suite (e.g., AES-CBC).
- **Legacy Risk:** SSL v2/v3 are broken; they lack Forward Secrecy.

The SSL RSA Handshake

Client Hello
Supported suites & random string

Server Hello
Chosen suite & Certificate (Pub Key)

Pre-Master Secret
Encrypted with Server's RSA Public Key

Session Keys
Both derive secret keys for Symmetrical Encryption

Transitioning to TLS

SSL Flaws

RSA Key Exchange: If the server's private key is ever stolen, **all past captured traffic** can be decrypted. This is a massive privacy risk for long-term data.

TLS Improvements

Enforces **Perfect Forward Secrecy (PFS)** via Diffie-Hellman. Each session generates a temporary key that is destroyed after use, protecting past data from future breaches.

Elliptic Curve (ECC) Mastery

The Modern Go-To Solution

Elliptic Curve Cryptography (ECC) uses the complex mathematics of curves rather than massive prime numbers (RSA).

Efficiency: A 256-bit ECC key is as strong as a 3072-bit RSA key.

Speed: Lower computational overhead means faster handshakes and less battery drain on mobile devices.

CIA Role: Enhances **Availability** by preventing CPU-exhaustion attacks.

Figure 11: *The elliptic curve* $y^2 = x^3 + x + 1 \pmod 5$ *over* $\mathbb{F}_{125}$ *(left),* $\mathbb{F}_{625}$ *(middle) and* $\mathbb{F}_{3125}$ *(right).*

Figure 12: *Points of* $E: \ y^2 = x^3 + 3 \pmod 7$ *over the fields* $\mathbb{F}_{343}$ *(left),* $\mathbb{F}_{2401}$ *(center) and* $\mathbb{F}_{16,807}$ *(right*

The TLS 1.3 Handshake

Hello + Key Share
Sends ECC Pub Key & Cipher list immediately

Server Key Share
Sends its ECC Pub Key & Hashed Certificate

Independent Derivation
Both derive secret keys locally (ECDHE)

1 Round Trip (1-RTT) Speed Achievement

Comparison: SSL vs. TLS

Feature	SSL (Legacy)	TLS (Current)
Port Strategy	Implicit (dedicated ports)	Explicit (STARTTLS)
Key Exchange	Primarily RSA	ECDHE (Diffie-Hellman)
Authentication	Handshake message only	HMAC-based hashing
Forward Secrecy	Rarely supported	Mandatory in TLS 1.3